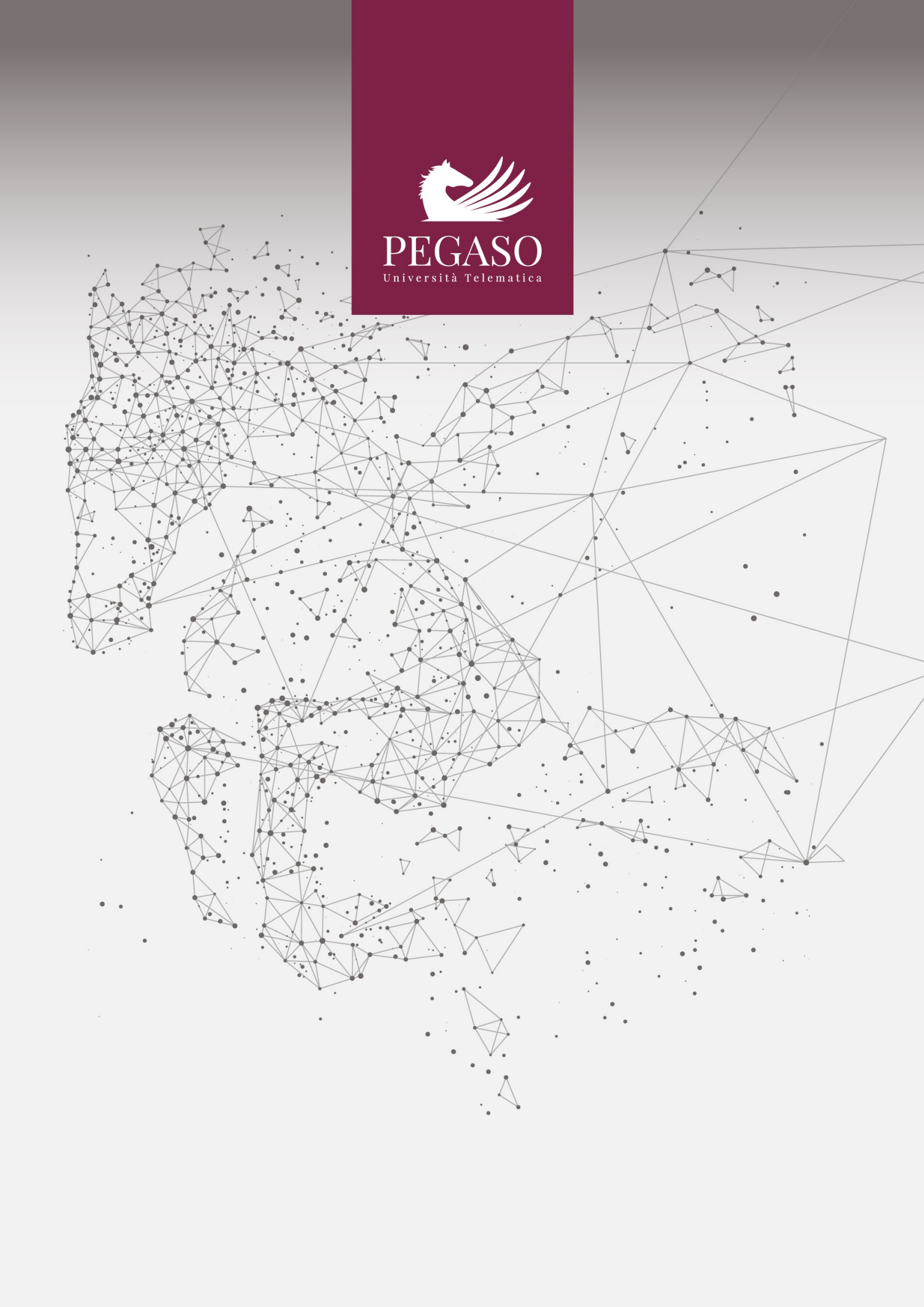

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3

2. GLOBBING DI NOMI DI FILE 4

2.1 ESERCITAZIONE CON GLOBBING ED ESPRESSIONI REGOLARI 5

3. SERIALIZZAZIONE DEI DATI 7

3.1 ESERCITAZIONE DI SERIALIZZAZIONE/DESERIALIZZAZIONE DI STRUTTURE DATI 9

3.1.1 Serializzazione e scrittura su file binario 9

3.1.2 Lettura da file binario e deserializzazione 11

3.1.3 Serializzazione/deserializzazione e scrittura/lettura su file binario 12

4. ESERCIZIO DELL'INDICIZZATORE DI FILE 15

BIBLIOGRAFIA 19

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si approfondiscono diversi concetti relativi ai file. In primo luogo, viene introdotto il globbing di nomi di file come sostituto delle espressioni regolari nell'analisi del file system. Viene, in seguito, illustrato il concetto di serializzazione e deserializzazione dei dati per il salvataggio dei file binari in memoria di massa. Infine, viene illustrato e analizzato un problema completo dal punto di vista delle conoscenze in Python: l'indicizzatore di file che presuppone conoscenze e abilità relative a liste, espressioni regolari, globbing, file, dizionari.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Globbing di nomi di file

Le espressioni regolari sono ottimi strumenti, ma in molti casi sono fin troppo potenti. Ad esempio, nel caso in cui si voglia individuare un nome di file in base all'estensione è possibile utilizzare il **globbing.** Il globbing è l'operazione che consiste nel **ricercare nomi di file e altre stringhe tramite espressioni regolari semplificate.**

Un carattere jolly può essere costituito dal segno asterisco * che rappresenta zero o più caratteri, o dal punto interrogativo ? che rappresenta esattamente un carattere (\ + . non sono codici speciali nel globbing).

Il modulo **glob** di Python offre la funzione **glob**() per l'uso dei caratteri jolly. La funzione glob() restituisce una lista di tutti i nomi di file individuati dal parametro passato. Nel caso successivo, si ricercano tutti i file con estensione .txt all'interno della cartella in cui è contenuto il file Python del programma.

```
import glob

print(glob.glob("*.txt"))
```

Il risultato dell'elaborazione precedente è una lista contenente tutti i nomi dei file con estensione .txt.

```
['LICENSE.txt', 'NEWS.txt']
```

La funzione **iglob**() funziona come glob(), ma restituisce un iteratore che non memorizza simultaneamente tutti i valori restituiti da glob(). Tale funzionalità risulta essere molto utile nel caso di elaborazioni computazionalmente molto pesanti.

Molto importante è, anche, la possibilità di passare la stringa pattern semplificata indicando anche il **percorso (path)** della cartella in cui ricercare.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per file e directory, ad esempio nella cartella C è possibile utilizzare il codice seguente:

```
import glob
ris = glob.iglob('C:/*')
for a in ris:
    print(a)
```

Per i soli file (no cartelle) ad esempio nella cartella C è possibile, invece, usare il codice seguente:

```
import glob
ris = glob.iglob('C:/*.*')
for a in ris:
    print(a)
```

Python mette a disposizione con il metodo glob, la possibilità di **ricorsione** per la ricerca. In particolare, la ricerca avviene in un **sottoalbero del filesystem** partendo dalla cartella in cui è presente il file. Nel codice seguente si cercano tutti i file di testo:

- prima nella directory in cui presente il file;
- poi si sale di un livello (nella directory che contiene la directory in cui si trova il file) e si verificano tutte le directory a cascata verso il basso;
- si ripete la stessa operazione salendo ancora di un altro livello.

Essendo computazionalmente pesante, è consigliato l'uso della funzione iglob() al posto di glob().

```
import glob
ris = glob.iglob('**/*.txt', recursive=True)
for a in ris:
    print(a)
```

2.1 Esercitazione con globbing ed espressioni regolari

È possibile anche procedere con l'uso combinato di:

- globbing di nomi di file per ottenere l'elenco dei file;
- funzione findall() del modulo re da utilizzare con i file.

Si vuole ricercare l'elenco degli articoli della Legge 240 contenuta in un file di testo. Innanzitutto, è necessario conoscere il nome del file (ottenibile con globbing e non direttamente con il nome del file).

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Successivamente:

- si apre il file in lettura;
- si cercano attraverso un'espressione regolare, la funzione di lettura del file e la funzione findall() del modulo re, tutti gli articoli nella forma Art. X.

```
import glob
import re

lista_file = glob.glob("*.txt")

for testo in lista_file:
    infile = open(testo, 'r')
    ris = re.findall(r"art. \d+", infile.read(), re.I)
    print("Lista degli articoli nella legge 240")
    for a in ris:
        print(a)
    infile.close()
```

Il risultato dell'elaborazione del programma precedente applicato nella cartella contenente il file txt è il seguente:

```
Lista degli articoli nella legge 240
Art. 1
Art. 2
Art. 3
Art. 4
Art. 5
Art. 6
Art. 7
Art. 8
Art. 9
Art. 10
Art. 11
Art. 12
Art. 13
Art. 14
Art. 15
Art. 16
Art. 17
Art. 18
Art. 19
Art. 20
Art. 21
Art. 22
Art. 23
Art. 24
Art. 25
Art. 26
Art. 27
Art. 28
Art. 29
```

PEGASO
Università Telematica

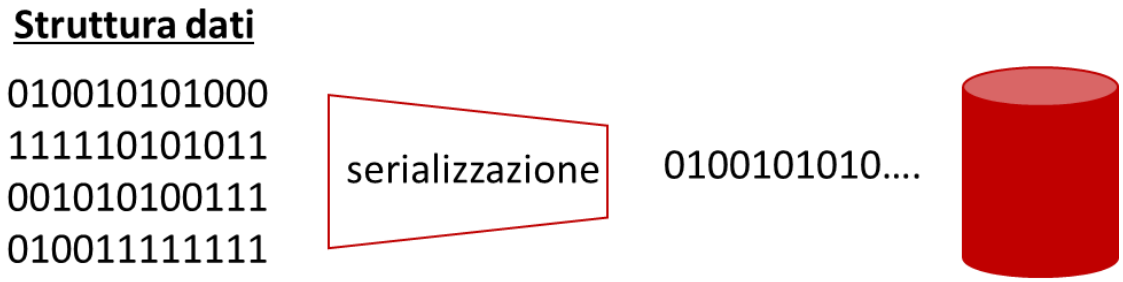
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Serializzazione dei dati

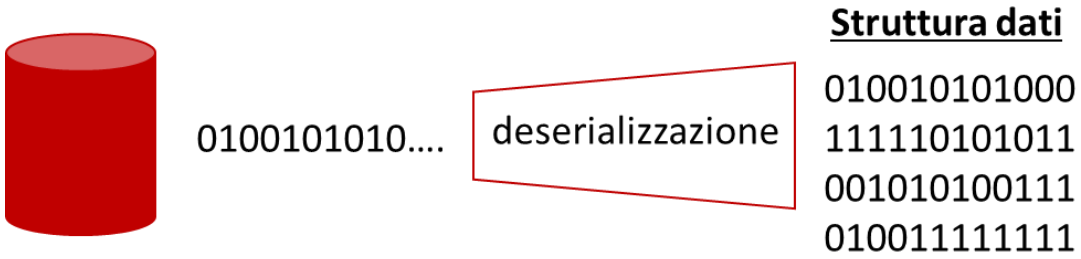
Nell'ambito della memorizzazione dei dati, finora abbiamo lavorato solo con i **file di testo.**

La lettura/scrittura di file di testo è complessa quando si vogliono leggere/salvare su disco le strutture dati come liste, set, dizionari, ecc. Per tale scopo è utile usare i **file binari.**

Per poter salvare un oggetto in un file è necessario, però, **serializzarlo.** L'**oggetto serializzato** è un oggetto convertito in un flusso di byte che possono essere salvati in un file per poter essere recuperate in un secondo momento. Dopo la serializzazione è possibile salvare la sequenza su disco. La figura seguente riporta uno schema semplificato della serializzazione dei dati e della loro memorizzazione in memoria di massa.



Per leggere l'oggetto da disco si procede con la **deserializzazione** della sequenza. La figura seguente riporta uno schema semplificato della deserializzazione dei dati e della loro lettura dalla memoria di massa.



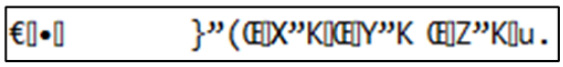
È molto importante sottolineare la differenza tra file di testo e file binari dal punto della **codifica utilizzata.** Se si prova ad aprire un file binario con un editor di testo, essendo la **codifica non testuale**, la sequenza di bit verrà erroneamente interpretata come sequenza di caratteri con codifica Unicode (come mostrato nella figura seguente).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Struttura dati

010010101000
111110101011
001010100111
010011111111



La libreria standard di Python dispone di un modulo denominato **pickle** che contiene diverse funzioni per la serializzazione degli oggetti (pickling in italiano vuol dire mettere sottaceto i cetriolini).

Dopo aver importato il **modulo pickle** (import pickle):

- si apre file in modalità binaria (in scrittura in questo caso);
- si chiama il metodo dump() del modulo pickle per serializzare l'oggetto e scriverlo nel file specificato;
- si chiude il file.

Il codice che permette di scrivere su file binario una struttura dati (es. dizionario) è il seguente:

```
import pickle

dizionario = {'X': 1, 'Y': 2, 'Z': 3}

outputfile = open('dizionario.dat','wb')
pickle.dump(dizionario,outputfile)
outputfile.close()
```

Il codice che permette di leggere da file binario una struttura dati (es. dizionario salvato in precedenza), invece, è il seguente:

```
import pickle

inputfile = open('dizionario.dat','rb')
dizionario = pickle.load(inputfile)
inputfile.close()
print(dizionario)
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il risultato della lettura del dizionario da file binario è il seguente:

```
{'X': 1, 'Y': 2, 'Z': 3}
```

3.1 Esercitazione di serializzazione/deserializzazione di strutture dati

3.1.1 Serializzazione e scrittura su file binario

Si vuole scrivere un programma che chiede all'utente di inserire le seguenti informazioni:

- nominativo;
- numero di telefono;
- città;

riguardanti un certo numero di persone.

Ogni volta che l'utente inserisce queste informazioni, le stesse verranno memorizzate in un dizionario. Al termine della fase di inserimento dei dati, il dizionario verrà salvato su file binario. Per segnalare la terminazione dell'inserimento di informazioni sulle persone, verrà utilizzata una **sentinella**. La sentinella è un valore che permette di terminare un ciclo di cui non è noto il numero di iterazioni. In questo caso la sentinella sarà un carattere diverso da 's' o 'S' che indica Sì.

La funzione salva_dati() permette di inserire i dati, memorizzarli in un dizionario e salvare tali informazioni in memoria di massa. La funzione main() permette di ripetere la funzione salva_dati() finchè non verrà utilizzata una sentinella.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import pickle


def salva_dati(file):
    persona = dict()
    persona["nome"] = input("Inserire nome: ")
    persona["numero"] = input("Inserire numero: ")
    persona["città"] = input("Inserire città: ")
    pickle.dump(persona,file)


def main():

    ripeti = 's'
    outputfile = open("info.dat","wb")
    while ripeti.lower() == 's':
        salva_dati(outputfile)
        ripeti = input("Vuoi inserire altri dati? (s/n) ")
    outputfile.close()


main()
```

Il risultato dell'elaborazione del programma precedente per nomi di prova è il seguente:

```
Inserire nome: Paolo Rossi
Inserire numero: 123
Inserire città: Roma
Vuoi inserire altri dati? (s/n) s
Inserire nome: Giovanni Verdi
Inserire numero: 456
Inserire città: Milano
Vuoi inserire altri dati? (s/n) s
Inserire nome: Rosa Bianchi
Inserire numero: 789
Inserire città: Firenze
Vuoi inserire altri dati? (s/n) n
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3.1.2 Lettura da file binario e deserializzazione

Si vuole scrivere un programma che carica da file binario e visualizzare a schermo le informazioni delle persone scritte con il programma precedente. Verranno utilizzate due funzioni:

- visualizza_dati() che permette di visualizzare in modo corretto le informazioni relative ad un singolo dizionario (corrispondente ad una persona);
- main() che apre il file binario in lettura e legge le informazioni scritte in memoria con il programma precedente.

```
import pickle

def visualizza_dati(p):
    print("Nome:",p["nome"])
    print("Numero:",p["numero"])
    print("Città:",p["città"])
    print()


def main():
    try:
        print("Informazioni sulle persone")
        end_of_file = False
        infile = open('info.dat','rb')
        while not end_of_file:
            try:
                persona = pickle.load(infile)
                visualizza_dati(persona)
            except EOFError:
                end_of_file = True
                infile.close()
    except FileNotFoundError:
        print("Dati non disponibili")


main()
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il risultato dell'elaborazione del programma di lettura da file binario è il seguente:

```
Informazioni sulle persone
Nome: Paolo Rossi
Numero: 123
Città: Roma

Nome: Giovanni Verdi
Numero: 456
Città: Milano

Nome: Rosa Bianchi
Numero: 789
Città: Firenze
```

3.1.3 Serializzazione/deserializzazione e scrittura/lettura su file binario

Il problema principale dei due esercizi precedenti è che non è stata utilizzata una **struttura dati idonea** affinché il programma in fase di avvio possa caricare tutte le informazioni da file.

Modifichiamo, dunque, in modo opportuno il caso precedente, prevedendo una **lista di dizionari** dove ciascun dizionario corrisponde ad una persona. In questo caso, le informazioni verranno caricate da file all'avvio del programma se presenti in memoria.

Quando il programma verrà terminato, **il dizionario eventualmente integrato sarà scritto su file.**

La funzione main inizializza una lista vuota denominata lista_persone. Successivamente, richiama le funzioni leggi_da_file() e visualizza_dati() per recuperare (se esistenti) le informazioni delle persone già presenti in memoria di massa. Viene gestita anche l'eccezione FileNotFoundError nel caso in cui il file da aprire in lettura non esistesse in memoria. Si procede, quindi, all'inserimento di nuove persone attraverso la ripetizione interrotta dalla sentinella. Per ogni persona da inserire, verrà richiamata la funzione salva_persona() a cui verrà passata la lista di dizionari (persone) per permettere l'aggiunta della persona stessa.

Al termine del ciclo di inserimento, tramite la funzione salva_su_file(), è possibile salvare in memoria di massa in una sola volta tutta la struttura dati (lista di dizionari).

Si riporta il codice sorgente della funzione main():

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def main():
    filename = "persone.dat"
    lista_persone = list()
    try:
        print("Informazioni sulle persone")
        lista_persone = leggi_da_file(filename)
        visualizza_dati(lista_persone)
    except FileNotFoundError:
        print("Dati non disponibili")

    print("Inserimento nuove persone")
    ripeti = 's'
    while ripeti.lower() == 's':
        salva_persona(lista_persone)
        ripeti = input("Vuoi inserire altri dati? (s/n) ")
    salva_su_file(filename,lista_persone)


main()
```

Si riporta il codice sorgente delle funzioni:

- leggi_da_file() che prende come parametro il nome del file da aprire in lettura e restituisce la lista di dizionari;
- visualizza_dati() che prende come parametro la lista delle persone e la visualizza a schermo;
- salva_persona() che prende come parametro la lista delle persone, la aggiorna e la restituisce tramite append di un nuovo dizionario relativo ad una nuova persona;
- salva_su_file() che prende come parametri il nome del file da aprire in scrittura e la lista delle persone e salva su file la lista di dizionari.

```
def leggi_da_file(nomefile):
    infile = open(nomefile,'rb')
    lista_persone = pickle.load(infile)
    infile.close()
    return lista_persone

def visualizza_dati(lista_persone):
    for elem in lista_persone:
        print("Nome:",elem["nome"])
        print("Numero:",elem["numero"])
        print("Città:",elem["città"])
        print()
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def salva_persona(lista_persone):
    persona = dict()
    persona["nome"] = input("Inserire nome: ")
    persona["numero"] = input("Inserire numero: ")
    persona["città"] = input("Inserire città: ")
    lista_persone.append(persona)
    return lista_persone

def salva_su_file(nomefile, lista_persone):
    outputfile = open(nomefile,"wb")
    pickle.dump(lista_persone,outputfile)
    outputfile.close()
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Esercizio dell'indicizzatore di file

L'**indicizzatore di file**[1] è un programma che indicizza tutti i file da una specifica cartella indicata dall'utente. Il programma deve costruire un dizionario in cui:

- le chiavi sono tutte le parole univoche di tutti i file descritte dall'espressione regolare r"\w+" trattando le parole senza distinguere fra lettere maiuscole e minuscole;
- il valore di ogni voce è una lista di nomi di file che contengono tale parola.

Ad esempio, se la parola "ciao" è menzionata nei file a.txt e b.txt, il dizionario dovrà contenere la voce {…, 'ciao': ['a.txt', 'b.txt'], …}. In caso di ripetizione di una parola nello stesso file, il dizionario finale dovrà essere come l'esempio precedente e **non** come segue: {…, 'ciao': ['a.txt', 'a.txt', 'b.txt'], …}

Questo esercizio è molto completo per i temi visti finora. Sarà, infatti, necessario conoscere i seguenti argomenti per risolverlo: liste, espressioni regolari, globbing, file, dizionari.

Attraverso il globbing si potranno conoscere i nomi dei file nella cartella. Per ciascun file di testo della cartella si analizzerà ciascuna riga. Per ciascuna riga sarà necessario procedere ad un parsing delle parole.

Per ciascuna parola l'algoritmo dovrà:

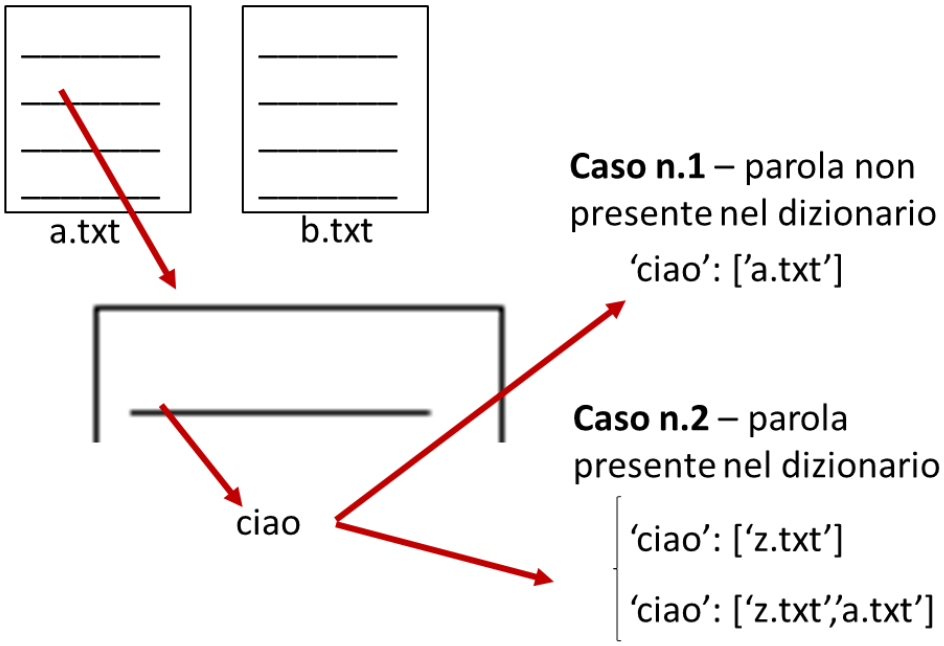
- aggiungere la parola come chiave nel dizionario (con valore associato il nome del file), se la parola non è presente nel dizionario;
- aggiungere alla lista dei valori associati a tale parola il nome del file nel caso in cui il nome dei file non sia già presente in tale lista.

La figura seguente schematizza il processo di analisi dei file, delle righe e delle parole presenti nel file stessi. Sono riportate anche le due casistiche possibili: assenza e presenza della parola nel dizionario.

[1] Traccia tratta da Dmitry Zinoviev, "Data Science con Python", Apogeo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



Analizziamo, ora, il codice sorgente del programma che permette di indicizzare i file.

In primo luogo, sarà necessaria l'importazione dei moduli glob e re. Successivamente si inizializzerà un dizionario vuoto e si procederà con il globbing dei file .txt (nel caso in esame i file sono a.txt, b.txt e c.txt; per analizzare tutti i file di una cartella basta modificare [abc].txt in *.txt).

Per ciascuno dei file contenuti in *lista_file* (risultato del globbing) si procederà con l'analisi. Il nome del file verrà scritto a schermo e si procederà con l'apertura del file di testo in lettura.

Per ciascuna riga del file di testo aperto in lettura, si provvederà a tokenizzare (suddividere in parole) la linea al fine di ottenere una lista di stringhe (parole) corrispondente alla riga stessa tramite la funzione findall() del modulo re.

```
import glob
import re

def main():
    d = dict()
    lista_file = glob.glob("[abc].txt")
    for nomefile in lista_file:
        print("Analisi del file: ", nomefile)
        infile = open(nomefile, 'r')
        for line in infile:
            ris = re.findall(r"\w+", line, re.I)
```

Successivamente, si analizza ciascuna parola presente nella lista secondo il seguente schema:

- se la parola trasformata in maiuscolo è presente nel dizionario ci sono due casi possibili:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. la parola è stata aggiunta al dizionario perché già presente nel file – non è richiesta nessuna azione;
2. la parola è stata aggiunta al dizionario perché presente in un altro file già analizzato – il nome del file corrente va aggiunto nella lista dei valori associati alla chiave (parola);
   si procede, quindi, recuperando il valore (lista di stringhe) associato alla chiave (parola.upper()).

Se il nome del file corrente non è presente nella lista vuol dire che ci si trova nel secondo caso e, quindi:

1. si inizializza una lista vuota;
2. si accoda il nome del file (per evitare che la stringa venga vista come lista di caratteri);
3. si concatena la lista delle stringhe con la lista con il nuovo nome del file;

- se la parola trasformata in maiuscolo non è presente nel dizionario:

1. si inizializza una lista vuota;
2. si accoda il nome del file (per evitare che la stringa venga vista come lista di caratteri);
3. si inserisce la coppia chiave (parola) – valore (lista con il nuovo nome del file) al dizionario.

```
            for parola in ris:
                if parola.upper() in d:
                    v = d[parola.upper()]
                    if nomefile not in v:
                        l = list()
                        l.append(nomefile)
                        d[parola.upper()]=d[parola.upper()]+l
                else:
                    l = list()
                    l.append(nomefile)
                    d[parola.upper()] = l
    infile.close()
```

Si procede, poi, con la visualizzazione del dizionario secondo il seguente codice sorgente:

```
        print()
        for parola in d:
            print(parola,":",d[parola])

    main()
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Utilizzando tre file di prova a.txt, b.txt e c.txt (a.txt e b.txt sono uguali) con parole ripetute come mostrato di seguito:

a.txt

```
ciao casa ciao albero ciao casa ciao albero
ciao casa ciao albero ciao casa ciao albero
ciao casa ciao albero ciao casa ciao albero
ciao casa ciao albero ciao casa ciao albero
ciao casa ciao albero ciao casa ciao albero
```

b.txt

```
ciao casa ciao albero ciao casa ciao albero
ciao casa ciao albero ciao casa ciao albero
ciao casa ciao albero ciao casa ciao albero
ciao casa ciao albero ciao casa ciao albero
ciao casa ciao albero ciao casa ciao albero
```

c.txt

```
albero
albero
albero
albero
albero
```

L'elaborazione del programma precedente risulta come segue:

```
Analisi del file:  a.txt
Analisi del file:  b.txt
Analisi del file:  c.txt

CIAO : ['a.txt', 'b.txt']
CASA : ['a.txt', 'b.txt']
ALBERO : ['a.txt', 'b.txt', 'c.txt']
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Zinoviev, D. (2019). Data Science con Python. Apogeo.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).